

TaskeHero

Complete Setup Guide

Production-Ready React Native (Expo) To-Do Reminder App
with Firebase Backend

Version 1.0

Table of Contents

1. Project Overview
2. Prerequisites
3. Firebase Setup Instructions
4. Installation Steps
5. Project Architecture and Folder Structure
6. Configuration Files
7. Firestore Security Rules
8. Running the App
9. Building for Production (APK/AAB)
10. Key Features Summary

1. Project Overview

TaskeHero is a production-ready to-do list and task reminder application built with React Native (Expo) and Firebase. The app provides a complete task management solution with authentication, real-time database synchronization, scheduled local notifications, and a polished modern user interface. It follows clean architecture principles and React best practices, including functional components with hooks, a service-based layer for Firebase operations, React Context for state management, and a reusable component library.

The app is designed for Android and supports email/password authentication through Firebase Auth, real-time task storage and retrieval using Firestore with subcollection-based user data isolation, and local push notifications via expo-notifications that trigger precisely at the user-selected date and time. Each task can be assigned a title, optional description, and an optional reminder with both date and time pickers. Tasks are organized into Pending and Completed sections with tab-based filtering, and a Floating Action Button provides quick access to task creation.

The entire codebase is organized into a clear folder structure with separate directories for screens, components, navigation, services, context, hooks, and utilities. This separation of concerns ensures scalability and maintainability. Form validation, error handling with user-friendly messages, loading spinners during API calls, confirmation alerts before destructive actions, and persistent login sessions via AsyncStorage are all built in.

2. Prerequisites

Before you begin setting up the TaskeHero project, ensure that all of the following tools and accounts are installed and configured on your development machine. Each prerequisite plays a critical role in the development, building, and deployment pipeline.

2.1 Required Software

Tool	Purpose	Source
Node.js (v18+)	JavaScript runtime required by Expo CLI and all build tools	nodejs.org
npm or yarn	Package manager for installing dependencies (npm ships with Node.js)	npmjs.com
Expo CLI	Command-line tool for creating and running Expo projects	expo.dev
Android Studio	Android emulator and SDK for testing on Android devices	developer.android.com

Android SDK (API 33+)	Required for building and running the app on Android	Via Android Studio
Firebase Account	Free account for Authentication, Firestore, and Cloud services	console.firebase.google.com
Git	Version control system for managing project files	git-scm.com

Table 1: Required development tools and sources

2.2 Environment Setup

Install Node.js from the official website or via your system package manager (e.g., `nvm` for version management). After installation, verify it by running **`node -v`** and **`npm -v`** in your terminal. Install the Expo CLI globally with **`npm install -g expo-cli`**. For Android development, install Android Studio, configure the Android SDK (minimum API level 33), and set up at least one virtual device (AVD) via the AVD Manager. Ensure the `ANDROID_HOME` environment variable points to your SDK location, and that `platform-tools` is in your system `PATH`.

3. Firebase Setup Instructions

Firebase serves as the complete backend for TaskeHero, handling authentication, real-time database operations, and serverless infrastructure. Follow these steps carefully to create and configure your Firebase project. Each step is essential for the app to function correctly.

Step 1: Create a Firebase Project

Go to **console.firebase.google.com** and click "Add Project". Enter a project name (e.g., "taskehero-app"), accept the terms, and click "Create Project". You can disable Google Analytics for this project as it is not required.

Step 2: Register an Android App

In the Firebase console, go to Project Settings (gear icon) and click the Android icon under "Your apps". Register your app with the package name **`com.taskehero.app`** (must match `app.json`). Download the **`google-services.json`** file and place it in the project root directory (same level as `package.json`).

Step 3: Enable Authentication

Navigate to "Authentication" in the left sidebar. Click "Get Started" and enable the **Email/Password** sign-in method. This allows users to register and log in with their email address and a password.

Step 4: Create Firestore Database

Navigate to "Firestore Database" and click "Create Database". Choose **Start in test mode** initially for development. Select a Firestore location closest to your users. After creating the database, go to the Rules tab and paste the security rules from the **`firestore.rules`** file included in this project. Then publish the rules.

Step 5: Get Web App Config

In Project Settings, scroll to "Your apps" and click the Web icon (</>) to add a web app. Register it with any nickname. After registration, you will see a **firebaseConfig** object with apiKey, authDomain, projectId, storageBucket, messagingSenderId, and appId. Copy these values into **src/services/firebase.js**, replacing the placeholder values.

3.1 Firebase Config Example

Replace the placeholder values in **src/services/firebase.js**:

```
const firebaseConfig = {
  apiKey: "AIzaSyD.....",
  authDomain: "taskehero-app.firebaseio.com",
  projectId: "taskehero-app",
  storageBucket: "taskehero-app.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abcdef123456",
};
```

4. Installation Steps

After completing the Firebase setup and ensuring all prerequisites are installed, follow these steps to get the TaskeHero app running on your local development environment. Each command should be run from the project root directory.

Step 1: Navigate to the Project

Open your terminal and navigate to the TaskeHero project directory:

```
cd TaskeHero
```

Step 2: Install Dependencies

Run the following command to install all required packages:

```
npm install
```

Step 3: Place google-services.json

Ensure the google-services.json file downloaded from Firebase is placed in the project root directory. This file is required for Firebase to authenticate your Android app and is referenced by the app.json config.

Step 4: Update Firebase Configuration

Open **src/services/firebase.js** and replace the placeholder config values with your actual Firebase web app credentials from the Firebase console.

Step 5: Start the Development Server

Launch the Expo development server:

```
npx expo start
```

Step 6: Run on Android

Press "a" in the Expo terminal to open the app in an Android emulator, or scan the QR code with the Expo Go app on a physical Android device.

5. Project Architecture and Folder Structure

The TaskeHero project follows a clean, modular architecture pattern that separates concerns across multiple layers. The service layer handles all Firebase interactions, the context layer manages global authentication state, the navigation layer controls screen routing, and the UI layer is split between reusable components and screen-specific code. This architecture ensures that each module has a single responsibility and can be tested, modified, or replaced independently.

5.1 Complete Folder Structure

Path	Description
App.js	Application entry point; wraps app with AuthProvider and registers notification listeners
package.json	Project dependencies and scripts configuration
app.json	Expo project configuration (name, icons, splash, Android settings)
babel.config.js	Babel transpiler configuration for JSX and ES6+ support
eas.json	EAS Build configuration for development, preview, and production builds
firestore.rules	Firebase Firestore security rules for data access control
google-services.json	Firebase Android configuration (downloaded from console)
src/screens/	Screen components: LoginScreen, SignupScreen, HomeScreen, AddTaskScreen, EditTaskScreen
src/components/	Reusable UI components: TaskItem, EmptyState, FAB, FormInput, LoadingSpinner
src/navigation/	React Navigation setup with auth-based routing (AuthStack vs MainStack)
src/services/	Firebase service modules: firebase.js, authService.js, taskService.js, notificationService.js
src/context/	React Context for global auth state (AuthContext with useAuth hook)
src/utills/	Utilities: theme.js (colors, spacing, shadows), constants.js, validators.js

Table 2: Complete project folder structure

6. Configuration Files

This section describes each configuration file in the project root, explaining its purpose and the key values you may need to customize for your specific deployment environment.

6.1 app.json (Expo Configuration)

The app.json file configures the Expo project settings including the app name ("TaskeHero"), slug identifier, version number, orientation lock (portrait), Android-specific settings such as the package name (com.taskehero.app), adaptive icon configuration, the google-services.json file reference, and plugins for expo-notifications and expo-font. The splash screen and app icon paths are also defined here. When customizing, ensure the package name matches what you registered in the Firebase console.

6.2 package.json (Dependencies)

The package.json declares all project dependencies. Key packages include: Expo SDK (~52.0), React Native 0.76.5, Firebase v11 for auth and Firestore, React Navigation v7 for screen routing, expo-notifications for local push notifications, react-native-date-picker for date/time selection UI, @react-native-async-storage/async-storage for persisting the auth session, and @expo/vector-icons for the Ionicons icon set. All dependencies are pinned to compatible versions to ensure build stability.

6.3 eas.json (Build Configuration)

The eas.json file defines three build profiles: development (with dev client for hot reloading), preview (internal distribution as APK for testing), and production (AAB format for Google Play Store submission). You can customize these profiles to add environment variables, change build types, or configure signing certificates.

7. Firestore Security Rules

Firestore security rules are critical for protecting user data. The rules included in the firestore.rules file implement a user-isolation pattern where each user can only read and write their own tasks. Tasks are stored under the path /users/{userId}/tasks/{taskId}, and the rules verify that the authenticated user (request.auth.uid) matches the userId path segment. The create rule additionally validates that required fields (title as a non-empty string, completed as a boolean) are present. All other paths are denied by default.

7.1 Rules Explanation

Rule	Condition	Purpose
read, write	request.auth.uid == userId	Only the task owner can read/write their tasks

create	title is string, title.size() > 0	New tasks must have a non-empty title
create	completed is bool	Completed status must be a boolean value
delete	request.auth.uid == userId	Only the owner can delete their tasks

Table 3: Firestore security rules breakdown

7.2 Deploying Rules

To deploy the rules, open the Firebase console, navigate to Firestore Database, click the Rules tab, paste the entire contents of the `firestore.rules` file into the editor, and click "Publish". Alternatively, install the Firebase CLI with **`npm install -g firebase-tools`**, run **`firebase login`**, and use **`firebase deploy --only firestore:rules`** from the project root to deploy directly from the command line. Always test rules in the Firebase console simulator before publishing to production.

8. Running the App

After completing all setup steps, you can run the TaskeHero app in development mode. The development server supports hot reloading, meaning any code changes will be reflected immediately in the running app without a full rebuild.

8.1 Development Mode

Start the Expo development server by running **`npx expo start`** from the project root. This opens the Expo DevTools in your browser and displays a QR code. To run on an Android emulator, press "a" in the terminal. To run on a physical device, install the Expo Go app from the Play Store and scan the QR code. Ensure your device and development machine are on the same Wi-Fi network. Debugging can be done using React DevTools and the built-in Expo debugging menu (shake the device or press `Ctrl+M` on emulator).

8.2 Common Issues and Fixes

Issue	Solution
Firebase connection error	Verify <code>firebaseConfig</code> values and <code>google-services.json</code> placement
Notifications not working on emulator	Test on a physical device; emulators may not support all notification features
Build fails with duplicate modules	Clear cache: <code>npx expo start -c</code> and delete <code>node_modules</code> , then <code>npm install</code>
Auth state not persisting	Ensure <code>@react-native-async-storage/async-storage</code> is installed
Date picker not showing	Ensure <code>react-native-date-picker</code> is in dependencies

Table 4: Common development issues and solutions

9. Building for Production (APK/AAB)

When you are ready to distribute the app, you can build a production APK for direct installation or an AAB (Android App Bundle) for Google Play Store submission. TaskeHero uses EAS Build (Expo Application Services) for cloud-based builds, which eliminates the need for local Android SDK setup for production builds.

9.1 Building a Preview APK

To generate a preview APK for internal testing and distribution:

```
eas build --platform android --profile preview
```

This command uploads your project to EAS Build servers, compiles the APK using the preview profile defined in eas.json (which outputs APK format for easy sideloading), and provides a download link when the build completes. The preview profile is configured for internal distribution and does not require app signing setup.

9.2 Building for Google Play Store

To generate a production AAB for Play Store submission:

```
eas build --platform android --profile production
```

The production profile generates an AAB file optimized for Google Play delivery. Before your first production build, you need to generate a keystore for app signing. Run **eas credentials** to manage your keystore. EAS can generate and manage the keystore for you (recommended) or you can provide your own. After the build completes, download the AAB and upload it to the Google Play Console under your app release.

10. Key Features Summary

The following table summarizes all the major features implemented in TaskeHero, along with the technology or approach used for each feature. This serves as a quick reference for understanding the full capabilities of the application.

Feature	Technology / Approach
Email/Password Authentication	Firebase Auth with createUserWithEmailAndPassword and signInWithEmailAndPassword
Persistent Login Session	AsyncStorage via getReactNativePersistence for cross-restart session persistence

Real-time Task Sync	Firestore onSnapshot listener for live data synchronization
User Data Isolation	Subcollection pattern /users/{uid}/tasks/ with Firestore rules enforcement
Local Push Notifications	expo-notifications with SchedulableTriggerInputTypes.DATE for exact-time delivery
Date and Time Picker	react-native-date-picker with modal mode for native date/time selection
Task CRUD Operations	Firestore addDoc, updateDoc, deleteDoc with async/await error handling
Form Validation	Custom validator functions (email, password, task title, date-time)
Tab-based Filtering	All/Pending/Completed tabs with dynamic task list filtering
Loading Spinners	Dedicated LoadingSpinner component shown during all async operations
Delete Confirmation	React Native Alert with destructive action confirmation dialog
Empty State Design	Reusable EmptyState component with context-aware icons and messages
Clean Architecture	Separation of screens, components, services, context, navigation, and utils

Table 5: Complete feature matrix with technology details